МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»

Анализ сложности сортировок, не использующих сравнение элементов

ОТЧЁТ

студента 2 курса 211 группы
Катушов Егор Степанович

Проверено:
доцент, к. ф.-м. н.
М. И. Сафрончик

Саратов 2025

1 Сортировка 1: Сортировка подсчетом (Counting Sort)

Алгоритм сортировки подсчетом предназначен для целочисленных массивов. Он не использует сравнение элементов между собой, вместо этого опирается на подсчёт количества вхождений каждого значения.

1.1 Принцип работы

1. Находится максимальный элемент в массиве для определения диапазона значений.
2. Создаётся вспомогательный массив-счётчик размером $k + 1$, где k — максимальное значение.
3. Первый проход по исходному массиву заполняет счётчики: для каждого элемента увеличивается соответствующая ячейка.
4. Строится массив префиксных сумм, который показывает, сколько элементов меньше или равно текущему значению.
5. Элементы расставляются в выходной массив в правильном порядке (обход выполняется с конца, чтобы сохранить устойчивость сортировки).
6. Результат копируется обратно в исходный массив.

1.2 Реализация с оценкой сложности

Листинг 1: Сортировка подсчетом

```cpp
void counting_sort(int arr[], int n) {                  // O(1)
    if (n <= 0) return;                                 // O(1)
    int k = arr[0];                                     // O(1)
    for (int i = 1; i < n; i++)                         // O(n)
        if (arr[i] > k) k = arr[i];                     // O(1)

    int* count = new int[k + 1]();                      // O(k)
    for (int i = 0; i < n; i++)                         // O(n)
        count[arr[i]]++;                                // O(1)
    for (int i = 1; i <= k; i++)                        // O(k)
        count[i] += count[i - 1];                       // O(1)

    int* ans = new int[n];                              // O(n)
    for (int i = n - 1; i >= 0; i--) {                  // O(n)
        ans[count[arr[i]] - 1] = arr[i];                // O(1)
        count[arr[i]]--;                                // O(1)
    }
    for (int i = 0; i < n; i++)                         // O(n)
        arr[i] = ans[i];                                // O(1)

    delete[] count;                                     // O(1)
    delete[] ans;                                       // O(1)
}
```

1.3 Правило суммы

В сортировке подсчетом все циклы идут **последовательно** (друг за другом, а не внутри друг друга). По **правилу суммы** складываем все сложности:

$$T(n,k) = O(1) + O(n) + O(k) + O(n) + O(k) + O(n) + O(n) = O(n+k)$$

1.4 Итоговая сложность

$$\boxed{T(n,k) = O(n+k)}$$

где n — количество элементов, k — максимальное значение в массиве.

1.5 Где и почему используется сортировка подсчетом

Где используется:

- Сортировка оценок студентов (от 0 до 100 баллов)
- Сортировка возраста людей (от 0 до 150 лет)
- Сортировка символов по кодам ASCII (0-255)
- В системах массовой обработки данных с ограниченным диапазоном

Почему используется:

- Работает за линейное время $O(n+k)$, что быстрее $O(n \log n)$
- Проста в реализации
- Устойчива (сохраняет порядок равных элементов)
- Идеальна, когда диапазон значений k невелик и сравним с n

2 Сортировка 2: Поразрядная сортировка (Radix Sort LSD)

Поразрядная сортировка обрабатывает числа по отдельным разрядам, начиная с младшего (LSD — Least Significant Digit). Для сортировки по каждому разряду используется устойчивая сортировка подсчетом.

2.1 Принцип работы

1. Находится максимальный элемент в массиве для определения количества разрядов.
2. Для каждого разряда (единицы, десятки, сотни и т.д.) выполняется устойчивая сортировка подсчетом.
3. На каждом шаге числа распределяются по 10 корзинам (от 0 до 9) в зависимости от значения текущего разряда.
4. После обработки всех разрядов массив оказывается полностью отсортированным.

2.2 Реализация с оценкой сложности

Листинг 2: Поразрядная сортировка

```
int getMax(int arr[], int n) {                          // O(1)
    int mx = arr[0];                                    // O(1)
    for (int i = 1; i < n; i++)                         // O(n)
        if (arr[i] > mx) mx = arr[i];                   // O(1)
    return mx;                                          // O(1)
}

void countSort_radix(int arr[], int n, int exp) {
    int* output = new int[n];                           // O(n)
    int count[10] = {0};                                // O(1)

    for (int i = 0; i < n; i++)                         // O(n)
        count[(arr[i] / exp) % 10]++;                   // O(1)
    for (int i = 1; i < 10; i++)                        // O(1)
        count[i] += count[i - 1];                       // O(1)
    for (int i = n - 1; i >= 0; i--) {                  // O(n)
        int digit = (arr[i] / exp) % 10;                // O(1)
        output[count[digit] - 1] = arr[i];              // O(1)
        count[digit]--;                                 // O(1)
    }
    for (int i = 0; i < n; i++)                         // O(n)
        arr[i] = output[i];                             // O(1)
    delete[] output;                                    // O(1)
}

void radix_sort(int arr[], int n) {
    int m = getMax(arr, n);                             // O(n)
    for (int exp = 1; m / exp > 0; exp *= 10)           // O(d)
        countSort_radix(arr, n, exp);                   // O(n)
}
```

2.3 Правило произведения

В поразрядной сортировке внешний цикл **вложен** относительно вызовов `countSort_radix`. По **правилу произведения**:

$$T(n, d) = O(n) + d \cdot O(n) = O(n \cdot d)$$

2.4 Итоговая сложность

$$\boxed{T(n, d) = O(n \cdot d)}$$

где n — количество элементов, d — количество разрядов ($d = \lfloor \log_{10}(max) \rfloor + 1$).

2.5 Где и почему используется поразрядная сортировка

Где используется:

- Сортировка больших массивов целых чисел (сотни миллионов записей)

- Сортировка строк фиксированной длины (например, номера телефонов, IP-адреса)
- В базах данных при сортировке по числовым ключам
- В системах обработки больших данных (Big Data)
- Для сортировки чисел с плавающей точкой (через преобразование в целые)

Почему используется:

- Работает за линейное время $O(n \cdot d)$, где d — константа для чисел фиксированной разрядности
- Не зависит от диапазона значений k, в отличие от сортировки подсчетом
- Требует всего $O(n)$ дополнительной памяти
- Эффективна для очень больших n (миллионы и миллиарды элементов)
- Устойчива, что важно при сортировке по нескольким ключам

Преимущество перед сортировкой подсчетом:

- Сортировка подсчетом при $k = 10^9$ потребовала бы 10^9 ячеек памяти
- Поразрядная сортировка для таких чисел выполнит всего $d = 10$ проходов

Таблица 1: Сравнение характеристик

Характеристика	**Counting Sort**	**Radix Sort**
Временная сложность	$O(n + k)$	$O(n \cdot d)$
Дополнительная память	$O(n + k)$	$O(n)$
Стабильность	Да	Да
Применяемое правило	Сумма	Произведение
Эффективен когда	k невелик	d фиксировано
Проблемный случай	$k \gg n$	Нет (d растёт логарифмически)
Класс сложности	P	P

3 Вывод

В ходе работы были проанализированы два алгоритма сортировки, не использующих сравнение элементов: сортировка подсчетом и поразрядная сортировка.

- **Сортировка подсчетом** имеет сложность $O(n + k)$. Все циклы идут последовательно, поэтому применяется **правило суммы**. Используется для сортировки целых чисел с небольшим диапазоном значений, где она работает за линейное время.
- **Поразрядная сортировка** имеет сложность $O(n \cdot d)$. Внешний цикл по разрядам вложен относительно вызовов сортировки подсчетом, поэтому применяется **правило произведения**. Используется для сортировки больших массивов целых чисел и строк фиксированной длины, решая проблему большого диапазона значений.

Оба алгоритма обходят теоретическую нижнюю оценку $\Omega(n \log n)$, характерную для сортировок, основанных на сравнениях. Это достигается за счёт использования специальных свойств входных данных — целочисленности и ограниченности диапазона или разрядности.